Technology Stack

Date	13 July 2026
Team ID	xxxxxx
Project Name	Comprehensive Measure of Well-being
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

Technology Stack Details

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	**Frontend / Client-Side** *(e.g., React, Angular, HTML/CSS)*	HTML5, CSS3, vanilla JavaScript, rendered via Flask/Jinja2 templates	Lightweight and dependency-free, so the analyst dashboard loads instantly without a build step. Jinja2 templating integrates directly with the Flask backend, keeping the form-driven UI simple to maintain.
2	**Backend / Server-Side** *(e.g., Node.js, Python/ Django, Java)*	Python (Flask)	Flask's minimal footprint suits a single-purpose prediction service, and its native Python runtime allows the trained scikit-learn model to be loaded and queried without any cross-language bridging.
3	**Database / Data Storage** *(e.g., MySQL, MongoDB, PostgreSQL)*	Flat-file storage – historical dataset (HDI.csv) and serialized model (HDI.pkl)	The source data is static and read-only, so a full relational database is unnecessary overhead. CSV and pickle files keep the prototype lightweight while still supporting fast pandas-based lookups.
4	**Cloud / Hosting / Deployment** *(e.g., AWS, Azure, Heroku, Docker)*	Local Flask development server, deployable to Render/ Heroku	Keeps hosting cost-free during development while remaining a one-step deploy to a managed platform such as Render or Heroku once the analytics tool is ready for wider access.
5	**Version Control & CI/ CD** *(e.g., GitHub, GitLab, Jenkins)*	Git & GitHub	Enables the team to track changes to the Flask app, training scripts, and model files, and provides a straightforward path to add automated checks before deployment.
6	**Third-Party APIs / Other Tools** *(e.g., Stripe, Twilio, Google Maps)*	scikit-learn & joblib/ pickle (model serialization)	Allows the pre-trained linear regression model to be serialized once and loaded instantly at request time, avoiding retraining and keeping prediction latency low.